

Experiment 02 · USART - Synchronous (Clocked) Serial Link

USART1 driving a data and a clock line, captured on the slave by SPI2 in slave mode

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

USART1 sync +
SPI2 slave
peripheral used

9.6 kHz bit clock
line settings

4 (TX, CK, ACK,
GND)
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards are made to talk to each other over **USART** only. The board programmed as **master** reads its blue B1 user button and measures how long it is held; the board programmed as **slave** has no button and only reacts to what arrives on the USART link, blinking its green LD2 LED in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Command transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	0xA1 CMD_SHORT	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	0xA2 CMD_LONG	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are single `#define`s at the top of the slave listing (`LONG_ON_MS` , `LONG_OFF_MS` , `LONG_REPEAT`), so the timing can be retuned without touching the protocol code.

1.2 Status indication on the master

Indication	When	Meaning
Blue	steady while the frame is on the wire	transmission in progress
Green + one 250 ms LD2 flash	after a valid acknowledgement	the slave received the command
Red + three fast LD2 flashes	acknowledgement missing or wrong	wiring, power or protocol fault

The RGB LED is optional - set `USE_RGB` to 0 in the master listing and the on-board green LD2 alone carries the same information. If your RGB LED is common-anode, set `RGB_COMMON_CATHODE` to 0 instead of rewiring.

1.3 Frame format

All four experiments in this series use the same three-byte payload so that the protocols can be compared fairly:

Byte	Value	Purpose
1	0xAA	sync pattern - alternating bits, easy to recognise on an oscilloscope and unlikely to appear in noise
2	0xA1 or 0xA2	the command itself
3	~cmd (0x5E / 0x5D)	one's-complement checksum; the receiver acts only if it matches

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one programmed as master, one as slave
1 set	Male-male jumper wires (Dupont)	keep them under ~20 cm
2	USB Mini-B cables	for programming, then for standalone power
1-2	USB power bank or 5 V phone charger	to run the boards away from the PC
1	Common-cathode RGB LED + 3 × 330 Ω	optional master status indicator
1	Small breadboard	only needed for the RGB LED / pull-up resistors

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PA9	USART1_TX	AF7, push-pull	CN10-21	D8	master: data out
PA8	USART1_CK	AF7, push-pull	CN10-23	D7	master: bit clock out
PB15	SPI2_MOSI	AF7→AF5, pull-up	CN10-26	–	slave: data in
PB13	SPI2_SCK	AF5, pull-down	CN10-30	–	slave: clock in
PC8	ACK handshake	GPIO in / out	CN10-2	–	slave drives, master reads
PC13	B1 user button	input, pull-up	CN7-23	–	master only
PA5	LD2 user LED	output, push-pull	CN10-11	D13	both boards
GND	0 V reference	–	CN10-20	–	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board, odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, identify a pin by its *name*, not by counting.

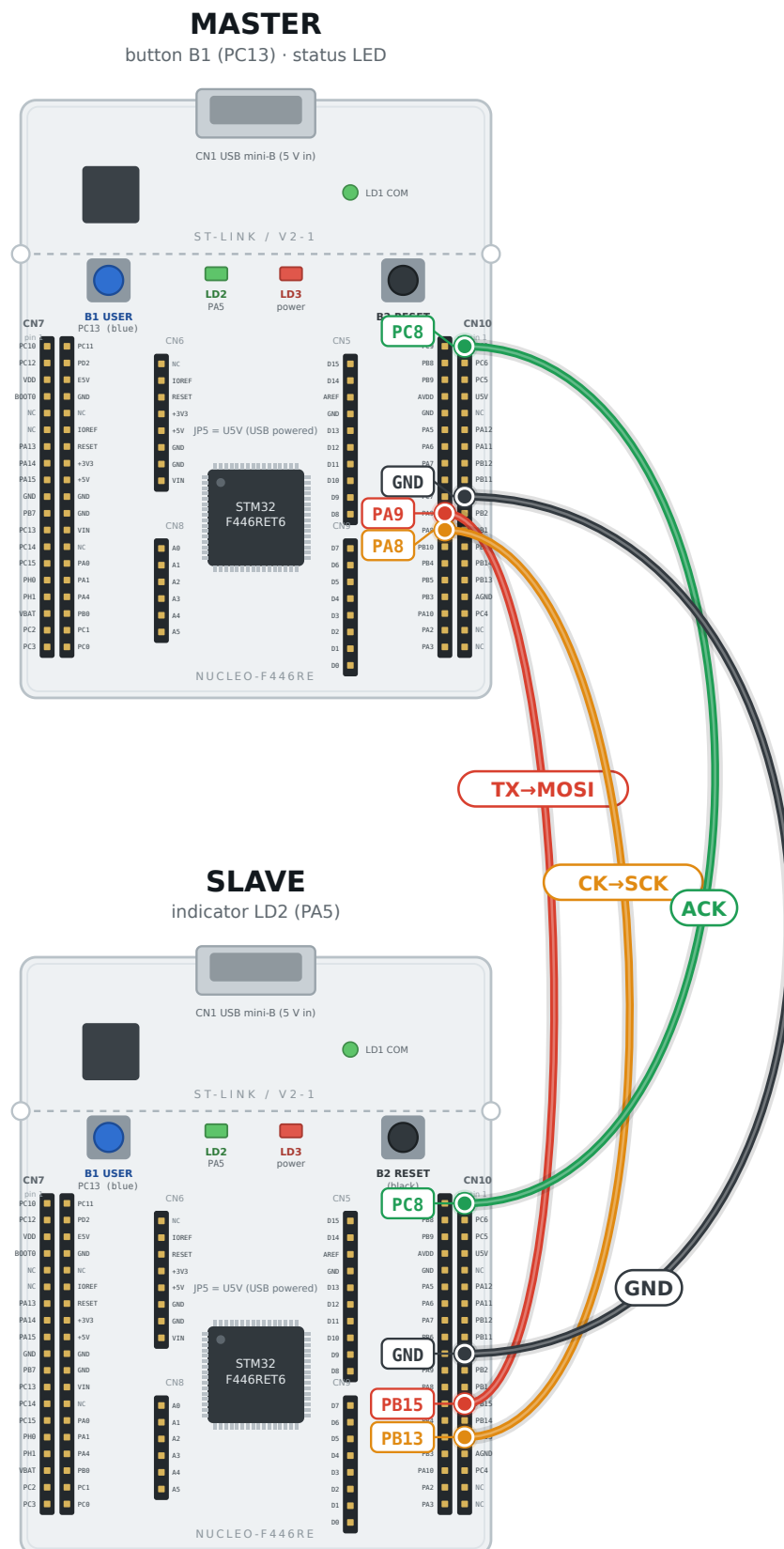
4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the last column are the ones used in Figure 1 on the next page; any colour works electrically, but keeping to a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Slave function	Wire colour used	Topology
Data	PA9 USART1_TX · CN10-21	PB15 SPI2_MOSI · CN10-26	data in	red	one direction
Clock	PA8 USART1_CK · CN10-23	PB13 SPI2_SCK · CN10-30	clock in	orange	one direction
ACK	PC8 input · CN10-2	PC8 output · CN10-2	handshake	green	slave → master
GND	GND · CN10-20	GND · CN10-20	GND	black	common

Experiment 2 · USART synchronous - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.



The master supplies the bit clock on PA8 (USART1_CK); the slave captures the clocked data with SPI2 in slave. PC8 is a plain GPIO handshake - the slave pulses it high for 10 ms to acknowledge, because USART_CK is out

Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

Setting `CR2.CLKEN` turns the USART into a **synchronous** transmitter: the peripheral now drives a bit clock on the `USART_CK` pin that is rigidly aligned with the data on `TX`. Three more bits shape that clock:

- **CPOL = 0** – the clock idles low between characters.
- **CPHA = 1** – data is captured on the second clock transition of each bit, i.e. the receiver samples in the middle of a stable bit.
- **LBCL = 1** – a clock pulse is also produced for the last (8th) data bit; without it the eighth bit would arrive unlocked.

RM0390 requires that `CLKEN`, `CPOL`, `CPHA` and `LBCL` are written while the transmitter is disabled, which is why the listing writes `CR1 = 0` first and only then sets `TE | UE`.

The fact that decides this experiment's architecture

On every STM32, `USART_CK` is an **output only**. The USART can be the synchronous *master* but never the synchronous *slave* – its receiver cannot be driven from an external clock. A second board therefore cannot receive this stream with its own USART in synchronous mode.

The clean solution, and the one used here, is to let the slave capture the clocked stream with **SPI2 in slave mode**, which is exactly a shift register driven by an external clock. Configured with `CPOL = 0`, `CPHA = 1` and `LSBFIRST = 1` it is bit-for-bit compatible with a USART synchronous character.

Why LSBFIRST matters

A USART always shifts the least-significant bit out first, while SPI defaults to most-significant first. Setting `SPI_CR1.LSBFIRST` on the slave makes the two agree; forget it and every received byte comes out bit-reversed (0xA1 would appear as 0x85).

Framing and the acknowledgement

`USART_CK` pulses only during the eight data bits – the start and stop bits carry no clock – so the SPI shift register sees exactly eight clocks per character and delivers one clean byte. Because the link is unidirectional, the slave answers on a separate GPIO: it pulses `PC8` high for 10 ms, and the master polls that pin for up to 500 ms. This is a perfectly ordinary hardware handshake and is worth pointing out in the report as the practical consequence of the master-only limitation above.

5.1 Clock configuration used by both boards

Both listings begin with `clock_init_hsi_16mhz()`, which selects the internal 16 MHz RC oscillator, leaves every bus prescaler at /1 and sets zero flash wait states. Nothing is left to a start-up file or to CubeMX, so **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** and every timing number in this manual can be verified by hand. `SysTick` is then reloaded with 15999 to interrupt once per millisecond, which is the time base for both the button measurement and the blink pattern.

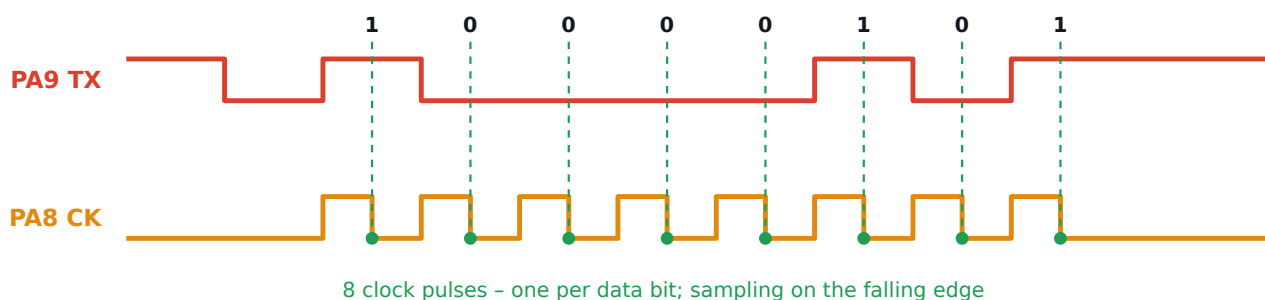
5.2 Register configuration summary

Register	Value written	Why
<code>RCC->APB2ENR</code>	<code> = USART1EN</code>	clock to USART1 (master)
<code>RCC->APB1ENR</code>	<code> = SPI2EN</code>	clock to SPI2 (slave)
<code>GPIOA->AFR[1]</code>	<code>PA8, PA9 = 7</code>	AF7 puts USART1_CK on PA8 and USART1_TX on PA9
<code>USART1->CR2</code>	<code>CLKEN CPHA LBCL</code>	enable the clock pin, sample on the 2nd edge, clock the last bit
<code>USART1->BRR</code>	<code>1667</code>	9600 bit/s, so CK runs at 9.6 kHz
<code>USART1->CR1</code>	<code>TE UE</code>	transmitter only – nothing is received on this link

GPIOB->AFR[1]	PB13, PB15 = 5	AF5 puts SPI2_SCK on PB13 and SPI2_MOSI on PB15
SPI2->CR1	CPHA LSBFIRST SSM SPE	slave (MSTR = 0), CPOL = 0, LSB first, internally selected
SPI2->SR	poll RXNE , OVR	one captured byte is ready / a byte was lost

6 Signal timing

USART1 synchronous mode - CPOL = 0, CPHA = 1, LBCL = 1



The clock idles low, rises at the start of each data bit and falls in its middle, where the receiver samples.
 START and STOP carry no clock pulse, so a shift-register receiver sees exactly 8 bits per character.

Figure 2 – Synchronous mode. PA8 now carries a bit clock that is generated by the transmitter itself, so the receiver no longer has to guess where the bit boundaries are.

7 MASTER source code `usart_sync_master.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the master project. It is self-contained: there are no other files to add and no header to edit.

`usart_sync_master.c`

```
1/*=====
2 * EXPERIMENT 2 : USART - SYNCHRONOUS MODE          --- M A S T E R ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 * WHAT IS DIFFERENT FROM EXPERIMENT 1
7 * Here USART1 runs in SYNCHRONOUS mode: CR2.CLKEN = 1 makes the peripheral
8 * output a bit clock on USART1_CK (PA8) that is strictly aligned with the
9 * data bits on USART1_TX (PA9). There is no baud-rate agreement problem
10 * any more - the receiver is clocked by the transmitter.
11 * CPOL = 0 (clock idles low), CPHA = 1 (data captured on the 2nd edge),
12 * LBCL = 1 (a clock pulse is also produced for the last = 8th data bit).
13 *
14 * IMPORTANT HARDWARE FACT (state this in your lab report)
15 * On STM32 the USART can only be the SYNCHRONOUS **MASTER** - USART_CK is
16 * an output-only pin, the receiver cannot be driven by an external clock.
17 * Therefore the slave board captures the clocked data with its SPI
18 * peripheral in slave mode (see the slave source file), and the return
19 * acknowledgement travels on a dedicated 1-wire handshake line PC8.
20 *
21 * WIRING (master -> slave)
22 * PA9 USART1_TX (CN10-21 / D8) -> PB15 SPI2_MOSI (CN10-26) data
23 * PA8 USART1_CK (CN10-23 / D7) -> PB13 SPI2_SCK (CN10-30) clock
24 * PC8 ACK-IN (CN10-2) <- PC8 ACK-OUT (CN10-2) handshake
25 * GND (CN10-20) <-> GND (CN10-20) common 0 V
26 *
27 * Optional RGB status LED on the MASTER (common cathode + 3 x 330 R):
28 * PC0 = RED (CN7-38), PC1 = GREEN (CN7-36), PC2 = BLUE (CN7-35).
29 *=====*/
30
31#include "stm32f446xx.h"
32#include <stdint.h>
33
34/*----- user configuration -----*/
35#define USE_RGB 1u
36#define RGB_COMMON_CATHODE 1u
37
38#define LED_PIN 5u /* PA5 - LD2 green user LED */
39#define BTN_PIN 13u /* PC13 - B1 blue user button */
40#define ACK_PIN 8u /* PC8 - handshake input from the slave */
41
42#define CMD_SYNC 0xAAu
43#define CMD_SHORT 0xA1u
44#define CMD_LONG 0xA2u
45
46#define T_DEBOUNCE_MS 20u
47#define T_LONG_MS 1000u
48#define T_ACK_MS 500u
49
50/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
51static void clock_init_hsi_16mhz(void)
52{
53    RCC->CR |= RCC_CR_HSION;
54    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
55
56    FLASH->ACR = FLASH_ACR_LATENCY_0WS;
57
58    RCC->CFGR &= ~RCC_CFGR_SW;
59    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
60
61    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
62}
63
```

```

64/*===== 2. SysTick 1 ms time base =====*/
65static volatile uint32_t s_ticks;
66
67void SysTick_Handler(void) { s_ticks++; }
68
69static void systick_init(void)
70{
71    SysTick->LOAD = 16000u - 1u;
72    SysTick->VAL = 0u;
73    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
74                  SysTick_CTRL_TICKINT_Msk |
75                  SysTick_CTRL_ENABLE_Msk;
76}
77static uint32_t millis(void) { return s_ticks; }
78static void delay_ms(uint32_t ms)
79{
80    uint32_t t0 = s_ticks;
81    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
82}
83
84/*===== 3. LD2, B1 and the ACK handshake pin =====*/
85static void led_init(void)
86{
87    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
88    (void)RCC->AHB1ENR;
89
90    GPIOA->MODER &= ~(3u << (LED_PIN * 2u));
91    GPIOA->MODER |= (1u << (LED_PIN * 2u));
92    GPIOA->OTYPER &= ~(1u << LED_PIN);
93    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
94    GPIOA->PUPDR &= ~(3u << (LED_PIN * 2u));
95    GPIOA->BSRR = (1u << (LED_PIN + 16u));
96}
97static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
98static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
99
100static void button_init(void)
101{
102    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
103    (void)RCC->AHB1ENR;
104
105    GPIOC->MODER &= ~(3u << (BTN_PIN * 2u));
106    GPIOC->PUPDR &= ~(3u << (BTN_PIN * 2u));
107    GPIOC->PUPDR |= (1u << (BTN_PIN * 2u));
108}
109static uint32_t button_down(void)
110{
111    return ((GPIOC->IDR & (1u << BTN_PIN)) == 0u);
112}
113static uint8_t button_capture(uint32_t *press_ms)
114{
115    uint32_t t0, d;
116
117    if (!button_down()) return 0u;
118    delay_ms(T_DEBOUNCE_MS);
119    if (!button_down()) return 0u;
120
121    t0 = millis();
122    while (button_down()) { __NOP(); }
123    delay_ms(T_DEBOUNCE_MS);
124
125    d = millis() - t0;
126    if (press_ms != 0) { *press_ms = d; }
127    return (d >= T_LONG_MS) ? CMD_LONG : CMD_SHORT;
128}
129
130/* PC8 = digital input with pull-DOWN; the slave drives it HIGH for ~10 ms. */
131static void ackpin_init(void)
132{
133    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
134    (void)RCC->AHB1ENR;
135}

```

```

136  GPIOC->MODER &= ~(3u << (ACK_PIN * 2u));    /* 00 = input          */
137  GPIOC->PUPDR &= ~(3u << (ACK_PIN * 2u));
138  GPIOC->PUPDR |= (2u << (ACK_PIN * 2u));    /* 10 = pull-down      */
139}
140static uint32_t ack_wait(uint32_t timeout_ms)
141{
142    uint32_t t0 = millis();
143    while ((uint32_t)(millis() - t0) < timeout_ms) {
144        if (GPIOC->IDR & (1u << ACK_PIN)) {
145            while (GPIOC->IDR & (1u << ACK_PIN)) { __NOP(); } /* eat pulse */
146            return 1u;
147        }
148    }
149    return 0u;
150}
151
152/*===== 4. Optional RGB indicator =====*/
153static void rgb_init(void)
154{
155    #if USE_RGB
156        uint32_t p;
157        RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
158        (void)RCC->AHB1ENR;
159        for (p = 0u; p <= 2u; p++) {
160            GPIOC->MODER &= ~(3u << (p * 2u));
161            GPIOC->MODER |= (1u << (p * 2u));
162            GPIOC->OTYPER &= ~(1u << p);
163            GPIOC->PUPDR &= ~(3u << (p * 2u));
164        }
165    #endif
166}
167static void rgb_set(uint32_t r, uint32_t g, uint32_t b)
168{
169    #if USE_RGB
170        #if RGB_COMMON_CATHODE
171            GPIOC->BSRR = (r ? (1u << 0) : (1u << 16)) |
172                (g ? (1u << 1) : (1u << 17)) |
173                (b ? (1u << 2) : (1u << 18));
174        #else
175            GPIOC->BSRR = (r ? (1u << 16) : (1u << 0)) |
176                (g ? (1u << 17) : (1u << 1)) |
177                (b ? (1u << 18) : (1u << 2));
178        #endif
179    #else
180        (void)r; (void)g; (void)b;
181    #endif
182}
183static void master_indicate(uint32_t ok)
184{
185    uint32_t i;
186    if (ok) {
187        rgb_set(0u, 1u, 0u);
188        led_on(); delay_ms(250u); led_off();
189    } else {
190        rgb_set(1u, 0u, 0u);
191        for (i = 0u; i < 3u; i++) { led_on(); delay_ms(60u); led_off(); delay_ms(60u); }
192    }
193    rgb_set(0u, 0u, 0u);
194}
195
196/*===== 5. USART1 in SYNCHRONOUS MASTER mode =====*/
197/* PA8 = USART1_CK (AF7)    PA9 = USART1_TX (AF7)
198 * BRR = 1667 -> 9600 bit/s, i.e. CK runs at 9.6 kHz.
199 * CR2.CLKEN / CPOL / CPHA / LBCL must be written while TE = 0 (RM0390). */
200static void usart1_sync_master_init(void)
201{
202    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
203    RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
204    (void)RCC->APB2ENR;
205
206    GPIOA->MODER &= ~((3u << (8u * 2u)) | (3u << (9u * 2u)));
207    GPIOA->MODER |= ((2u << (8u * 2u)) | (2u << (9u * 2u))); /* 10 = AF */

```



```

208  GPIOA->OTYPER  &= ~((1u << 8u) | (1u << 9u));
209  GPIOA->OSPEEDR |= ((3u << (8u * 2u)) | (3u << (9u * 2u)));
210
211  GPIOA->AFR[1] &= ~((0xFu << ((8u - 8u) * 4u)) | (0xFu << ((9u - 8u) * 4u)));
212  GPIOA->AFR[1] |= ((7u << ((8u - 8u) * 4u)) | (7u << ((9u - 8u) * 4u)));
213
214  USART1->CR1 = 0u;                                /* TE = 0 while configuring CR2 */
215  USART1->CR3 = 0u;
216  USART1->CR2 = USART_CR2_CLKEN |                  /* drive the CK pin */
217                USART_CR2_LBCL |                   /* clock pulse for the last bit */
218                USART_CR2_CPHA;                     /* CPHA = 1, CPOL = 0 */
219  USART1->BRR = 1667u;
220  USART1->CR1 = USART_CR1_TE | USART_CR1_UE;        /* transmit only */
221}
222static void usart1_write(uint8_t b)
223{
224    while ((USART1->SR & USART_SR_TXE) == 0u) { }
225    USART1->DR = (uint32_t)b;
226}
227static void usart1_flush(void)
228{
229    while ((USART1->SR & USART_SR_TC) == 0u) { }
230}
231
232/*===== main =====*/
233int main(void)
234{
235    uint8_t cmd;
236    uint32_t press_ms, ok;
237
238    clock_init_hsi_16mhz();
239    systick_init();
240    led_init();
241    button_init();
242    ackpin_init();
243    rgb_init();
244    usart1_sync_master_init();
245
246    rgb_set(0u, 0u, 1u); led_on(); delay_ms(300u); led_off(); rgb_set(0u,0u,0u);
247
248    for (;;) {
249        press_ms = 0u;
250        cmd = button_capture(&press_ms);
251        if (cmd == 0u) { continue; }
252
253        rgb_set(0u, 0u, 1u);
254
255        usart1_write(CMD_SYNC);
256        usart1_write(cmd);
257        usart1_write((uint8_t)(cmd ^ 0xFFu));
258        usart1_flush();
259
260        rgb_set(0u, 0u, 0u);
261
262        ok = ack_wait(T_ACK_MS);
263        master_indicate(ok);
264    }
265}

```

8 SLAVE source code `usart_sync_slave.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the slave project. It is self-contained: there are no other files to add and no header to edit.

`usart_sync_slave.c`

```
1/*=====
2 * EXPERIMENT 2 : USART - SYNCHRONOUS MODE          --- S L A V E ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 * WHY SPI IS USED ON THIS SIDE
7 *   An STM32 USART in synchronous mode can only be the clock MASTER; its
8 *   USART_CK pin is an output. To *receive* an externally clocked stream
9 *   the slave board uses SPI2 in slave mode, which is a pure shift register
10 *   driven by the incoming clock. Configuration that makes SPI2 byte-
11 *   compatible with a USART synchronous frame:
12 *
13 *       MSTR    = 0    slave, clock comes from outside
14 *       CPOL    = 0    idle low -> matches USART CPOL = 0
15 *       CPHA    = 1    2nd edge -> matches USART CPHA = 1
16 *       LSBFIRST = 1    the USART shifts the LSB out first
17 *       SSM = 1, SSI = 0 -> permanently "selected", no NSS wire needed
18 *       DFF     = 0    8-bit frames
19 *
20 *   USART_CK produces exactly 8 pulses per character (LBCL = 1 adds the
21 *   pulse for bit 7), so the SPI shift register delivers one clean byte per
22 *   character. The USART start/stop bits carry no clock and are ignored.
23 *
24 * WIRING
25 *   PB15 SPI2_MOSI (CN10-26) <- PA9 USART1_TX (master, CN10-21 / D8)
26 *   PB13 SPI2_SCK  (CN10-30) <- PA8 USART1_CK (master, CN10-23 / D7)
27 *   PC8 ACK-OUT   (CN10-2)  -> PC8 ACK-IN   (master, CN10-2)
28 *   GND           (CN10-20) <-> GND         (master, CN10-20)
29 *
30 * BLINK SPECIFICATION
31 *   CMD_SHORT (0xA1) -> 3 blinks, 150 ms ON / 150 ms OFF (300 ms spacing)
32 *   CMD_LONG  (0xA2) -> 3 blinks, 1000 ms ON / 1000 ms OFF (2 s spacing)
33 *=====*/
34
35#include "stm32f446xx.h"
36#include <stdint.h>
37
38#define LED_PIN          5u          /* PA5 - LD2 green user LED          */
39#define ACK_PIN          8u          /* PC8 - handshake output to the master */
40
41#define CMD_SYNC          0xAAu
42#define CMD_SHORT         0xA1u
43#define CMD_LONG          0xA2u
44
45#define SHORT_ON_MS       150u
46#define SHORT_OFF_MS      150u
47#define SHORT_REPEAT      3u
48#define LONG_ON_MS        1000u
49#define LONG_OFF_MS       1000u
50#define LONG_REPEAT       3u
51
52/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
53static void clock_init_hsi_16mhz(void)
54{
55    RCC->CR |= RCC_CR_HSION;
56    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
57
58    FLASH->ACR = FLASH_ACR_LATENCY_0WS;
59
60    RCC->CFGR &= ~RCC_CFGR_SW;
61    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
62
63    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
```

```

64}
65
66/*===== 2. SysTick 1 ms time base =====*/
67static volatile uint32_t s_ticks;
68
69void SysTick_Handler(void) { s_ticks++; }
70
71static void systick_init(void)
72{
73    SysTick->LOAD = 16000u - 1u;
74    SysTick->VAL = 0u;
75    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
76                  SysTick_CTRL_TICKINT_Msk |
77                  SysTick_CTRL_ENABLE_Msk;
78}
79static void delay_ms(uint32_t ms)
80{
81    uint32_t t0 = s_ticks;
82    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
83}
84
85/*===== 3. LD2 and the ACK pin =====*/
86static void led_init(void)
87{
88    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
89    (void)RCC->AHB1ENR;
90
91    GPIOA->MODER &= ~(3u << (LED_PIN * 2u));
92    GPIOA->MODER |= (1u << (LED_PIN * 2u));
93    GPIOA->OTYPER &= ~(1u << LED_PIN);
94    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
95    GPIOA->PUPDR &= ~(3u << (LED_PIN * 2u));
96    GPIOA->BSRR = (1u << (LED_PIN + 16u));
97}
98static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
99static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
100
101static void ackpin_init(void)
102{
103    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
104    (void)RCC->AHB1ENR;
105
106    GPIOC->MODER &= ~(3u << (ACK_PIN * 2u));
107    GPIOC->MODER |= (1u << (ACK_PIN * 2u)); /* push-pull output */
108    GPIOC->OTYPER &= ~(1u << ACK_PIN);
109    GPIOC->PUPDR &= ~(3u << (ACK_PIN * 2u));
110    GPIOC->BSRR = (1u << (ACK_PIN + 16u)); /* idle LOW */
111}
112static void ack_pulse(void)
113{
114    GPIOC->BSRR = (1u << ACK_PIN);
115    delay_ms(10u);
116    GPIOC->BSRR = (1u << (ACK_PIN + 16u));
117}
118
119static void run_pattern(uint8_t cmd)
120{
121    uint32_t i, n, on, off;
122
123    if (cmd == CMD_LONG) { n = LONG_REPEAT; on = LONG_ON_MS; off = LONG_OFF_MS; }
124    else { n = SHORT_REPEAT; on = SHORT_ON_MS; off = SHORT_OFF_MS; }
125
126    for (i = 0u; i < n; i++) {
127        led_on(); delay_ms(on);
128        led_off(); delay_ms(off);
129    }
130}
131
132/*===== 4. SPI2 as an externally clocked byte receiver =====*/
133/* PB13 = SPI2_SCK (AF5, input) PB15 = SPI2_MOSI (AF5, input) */
134static void spi2_capture_init(void)
135{

```

```

136 RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
137 RCC->APB1ENR |= RCC_APB1ENR_SPI2EN;
138 (void)RCC->APB1ENR;
139
140 GPIOB->MODER  &= ~( (3u << (13u * 2u)) | (3u << (15u * 2u)));
141 GPIOB->MODER  |=  ((2u << (13u * 2u)) | (2u << (15u * 2u))); /* I0 = AF */
142 GPIOB->OSPEEDR |=  ((3u << (13u * 2u)) | (3u << (15u * 2u)));
143
144 GPIOB->PUPDR  &= ~( (3u << (13u * 2u)) | (3u << (15u * 2u)));
145 GPIOB->PUPDR  |=  (2u << (13u * 2u)); /* SCK pull-DOWN: idles low */
146 GPIOB->PUPDR  |=  (1u << (15u * 2u)); /* MOSI pull-UP: UART idles hi */
147
148 GPIOB->AFR[1] &= ~( (0xFu << ((13u - 8u) * 4u)) | (0xFu << ((15u - 8u) * 4u)));
149 GPIOB->AFR[1] |=  ((5u << ((13u - 8u) * 4u)) | (5u << ((15u - 8u) * 4u)));
150
151 SPI2->CR1 = 0u;
152 SPI2->CR2 = 0u;
153 SPI2->CR1 = SPI_CR1_CPHA      | /* CPHA = 1, CPOL = 0 */
154             SPI_CR1_LSBFIRST | /* the USART sends the LSB first */
155             SPI_CR1_SSM;      /* SSM = 1, SSI = 0 -> always selected */
156 SPI2->CR1 |= SPI_CR1_SPE;
157}
158
159/* Non-blocking read of one captured byte. */
160static uint32_t spi2_get(uint8_t *b)
161{
162     if (SPI2->SR & SPI_SR_OVR) { /* clear a stale overrun */
163         (void)SPI2->DR; (void)SPI2->SR;
164     }
165     if (SPI2->SR & SPI_SR_RXNE) {
166         *b = *(volatile uint8_t *)&SPI2->DR;
167         return 1u;
168     }
169     return 0u;
170}
171
172/*===== main =====*/
173int main(void)
174{
175     uint8_t byte, cmd = 0u;
176     uint32_t state = 0u;
177
178     clock_init_hsi_16mhz();
179     systick_init();
180     led_init();
181     ackpin_init();
182     spi2_capture_init();
183
184     led_on(); delay_ms(300u); led_off(); /* "slave alive" signal */
185
186     for (;;) {
187         if (!spi2_get(&byte)) { continue; }
188
189         switch (state) {
190             case 0u:
191                 if (byte == CMD_SYNC) { state = 1u; }
192                 break;
193
194             case 1u:
195                 cmd = byte;
196                 state = 2u;
197                 break;
198
199             default:
200                 if ((uint8_t)(cmd ^ 0xFFu) == byte &&
201                     (cmd == CMD_SHORT || cmd == CMD_LONG)) {
202                     ack_pulse(); /* tell the master "got it" */
203                     run_pattern(cmd);
204                 }
205                 state = 0u;
206                 break;
207         }

```

```

208     }
209 }
210
211 /*=====
212 * APPENDIX - OPTION B : receive with the USART instead of SPI
213 * -----
214 * If your lab only requires that the CK line be *generated and observed*
215 * (e.g. on an oscilloscope) you may capture the data asynchronously instead.
216 * Wire master PA9 -> slave PA10 (keep PA8 -> a scope probe), then replace
217 * spi2_capture_init()/spi2_get() with:
218 *
219 *     static void usart1_rx_init(void)
220 *     {
221 *         RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
222 *         RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
223 *         GPIOA->MODER  &= ~(3u << (10u * 2u));
224 *         GPIOA->MODER  |=  (2u << (10u * 2u));
225 *         GPIOA->PUPDR  &= ~(3u << (10u * 2u));
226 *         GPIOA->PUPDR  |=  (1u << (10u * 2u));
227 *         GPIOA->AFR[1] &= ~(0xFu << ((10u - 8u) * 4u));
228 *         GPIOA->AFR[1] |=  (7u  << ((10u - 8u) * 4u));
229 *         USART1->CR1 = 0u; USART1->CR2 = 0u; USART1->CR3 = 0u;
230 *         USART1->BRR = 1667u;
231 *         USART1->CR1 = USART_CR1_RE | USART_CR1_UE;
232 *     }
233 *     static uint32_t usart1_get(uint8_t *b)
234 *     {
235 *         if (USART1->SR & USART_SR_ORE) { (void)USART1->SR; (void)USART1->DR; }
236 *         if (USART1->SR & USART_SR_RXNE) { *b = (uint8_t)USART1->DR; return 1u; }
237 *         return 0u;
238 *     }
239 *=====*/

```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** In the target selector open the *Board Selector* tab, type `NUCLEO-F446RE` and select it (or use the *MCU/MPU Selector* and pick `STM32F446RET6`).
2. Give the project a name, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and – this is the important one – set **Targeted Project Type = Empty**. An *Empty* project gives you the startup assembly file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is therefore nothing to configure graphically – which is exactly what a bare-metal experiment requires.
3. Answer **No** if the IDE offers to initialise peripherals; with an *Empty* project that dialog normally does not appear at all.
4. Open `Core/Src/main.c`, delete its whole contents and paste the listing from this manual. The single include `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the Empty project already put on the include path.
5. The symbol `STM32F446xx` is predefined by the project wizard. If the compiler complains that the header cannot decide which device it is, add it manually under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.
6. Build with **Ctrl + B**. Connect the first board, then *Run → Run As → STM32 C/C++ Application* to flash it.
7. Repeat steps 1–6 in a second project for the other listing, and flash it to the second board. Label the boards **MASTER** and **SLAVE** with a marker – they look identical once they are off the PC.

Alternative without the IDE: copy the generated `.bin` onto the `NODE_F446RE` mass-storage drive that appears when the board is plugged in, or use STM32CubeProgrammer.

10 Running the pair with no PC attached

Nothing in these programs depends on the debugger. The moment the board is powered, the reset vector in flash runs `main()`, so the two boards work as a stand-alone pair.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered USB hub. Leave jumper **JP5 on U5V** (its factory position).
3. LD3 (red) lights up on each board and both LD2s give a single ~300 ms flash: that is the power-up self-test built into the firmware.
4. Keep the protocol wires and the common ground connected exactly as during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into `E5V` (CN7 pin 6) and 0 V into `GND` (CN7 pin 8 or CN10 pin 20) of both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper of the protocol wiring becomes redundant – but leaving it in place does no harm.

Safety

Never drive 5 V into the `+5V` pin while JP5 is on U5V *and* a USB cable is plugged in – you would connect two supplies together. Choose one powering scheme and stay with it. BOOT0 must remain low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them exactly as in the connection table. Check the ground wire twice – it is the single most common cause of a dead link.
3. Power both boards. Each LD2 must give one short flash: both CPUs are alive.
4. **Test 1 – short click.** Press and release B1 on the master quickly (< 1 s). The master should flash green/LD2 and the slave should give three fast blinks.
5. **Test 2 – long press.** Hold B1 for at least one second, then release. The slave must now blink three times with one second on and one second off, i.e. two seconds between the start of one blink and the start of the next.
6. Time the slow pattern with a stopwatch and record it. Repeat each test three times.
7. **Test 3 – error case.** Pull the data wire out and press the button again. The master must show the red / triple-flash failure indication instead of green.
8. If an oscilloscope or logic analyser is available, capture the line(s) during one press and compare the capture with the timing figure in section 6.

Trial	Press duration (ms)	Expected command	Master indication	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
Every received byte is bit-reversed	<code>LSBFIRST</code> is not set on the slave's SPI2.
The slave reacts once and then never again	The SPI shift register lost its byte boundary. Press B2 RESET on the slave; if it happens repeatedly, shorten the clock wire and keep it away from the data wire.
Nothing at all is received, but a scope shows a clean clock	CK and MOSI are swapped, or the slave board is not powered. PA8 must go to PB13 and PA9 to PB15.
You only need to <i>observe</i> a synchronous clock	Use Option B in the appendix of the slave listing: the slave receives with its own USART asynchronously (master PA9 → slave PA10) while PA8 is probed with a scope.
Neither LD2 flashes at power-up	The board is not running: check LD3 (red) is lit, that the USB supply really delivers power, and that you flashed the board you think you flashed.
The slave never reacts, master always shows red	Ground is not common. Join GND (CN10-20) of the two boards.
Behaviour is erratic, blinks appear at random	Wires too long or badly seated. Use jumpers under 20 cm and push them fully home.
Both boards blink but the patterns are swapped	The master listing was flashed to the slave board. Label the boards.

Long press is detected as a short one

You released before 1000 ms. The threshold is `T_LONG_MS` in the master listing – change it there if your lab asks for a different value.

13 Review questions

1. Why can an STM32 USART be a synchronous master but not a synchronous slave?
2. What do CPOL and CPHA control, and which combination is used here?
3. What is the purpose of the LBCL bit?
4. Which bit makes SPI shift the LSB first, and why is it needed here?
5. How many clock pulses appear on CK for one 8-bit character, and why not ten?
6. How does synchronous mode remove the baud-rate tolerance problem?
7. Name two real devices that are driven with USART synchronous mode.
8. Why is the acknowledgement carried on a GPIO instead of the serial link?